
Gramática GLC

0)	programa'	→ programa
1)	programa	→ lista_funcoes
2)	lista_funcoes	→ lista_funcoes funcao
3)	lista_funcoes	→ funcao
4)	funcao	→ func nome_funcao (parametros) bloco
5)	nome_funcao	→ id
6)	nome_funcao	→ main
7)	parametros	→ lista_ids
8)	parametros	→ ε
9)	lista_ids	→ lista_ids , id
10)	lista_ids	→ id
11)	bloco	→ { lista_comandos }
12)	lista_comandos	→ lista_comandos comando
13)	lista_comandos	→ ε
14)	comando	→ declaracao ;
15)	comando	→ atribuicao ;
16)	comando	→ chamada ;
17)	comando	→ retorno ;
18)	comando	→ cmd_if
19)	comando	→ cmd_while
20)	comando	→ cmd_for
21)	comando	→ cmd_switch
22)	declaracao	→ tipo id
23)	declaracao	→ tipo id = expressao
24)	tipo	→ int
25)	tipo	→ float
26)	tipo	→ str
27)	tipo	→ bool
28)	atribuicao	→ id = expressao
29)	chamada	→ nome_chamada (argumentos)
30)	nome_chamada	→ id
31)	nome_chamada	→ puts
32)	nome_chamada	→ input
33)	argumentos	→ lista_expr
34)	argumentos	→ ε
35)	lista_expr	→ lista_expr , expressao
36)	lista_expr	→ expressao
37)	retorno	→ return expressao
38)	retorno	→ return
39)	cmd_if	→ if (expressao) bloco senao
40)	senao	→ else bloco
41)	senao	→ ε
42)	cmd_while	→ while (expressao) bloco
43)	cmd_for	→ for (for_ini ; expressao ; atribuicao) bloco
44)	for_ini	→ declaracao
45)	for_ini	→ atribuicao

46) cmd_switch → switch (expressao) { lista_casos }

47) lista_casos → lista_casos caso

48) lista_casos → ε

49) caso → case expressao : lista_comandos

--- Expressoes: precedencia embutida (menor -> maior) ---

50) expressao → expr_or

51) expr_or → expr_or OR expr_xor

52) expr_or → expr_xor

53) expr_xor → expr_xor XOR expr_and

54) expr_xor → expr_xor XNOR expr_and

55) expr_xor → expr_and

56) expr_and → expr_and AND expr_rel

57) expr_and → expr_rel

58) expr_rel → expr_rel == expr_ad

59) expr_rel → expr_rel != expr_ad

60) expr_rel → expr_rel < expr_ad

61) expr_rel → expr_rel > expr_ad

62) expr_rel → expr_rel <= expr_ad

63) expr_rel → expr_rel >= expr_ad

64) expr_rel → expr_ad

65) expr_ad → expr_ad + expr_mul

66) expr_ad → expr_ad - expr_mul

67) expr_ad → expr_mul

68) expr_mul → expr_mul * expr_pot

```

69)  expr_mul      → expr_mul / expr_pot
70)  expr_mul      → expr_mul % expr_pot
71)  expr_mul      → expr_pot

72)  expr_pot      → expr_prim ^ expr_pot      (assoc. a direita)
73)  expr_pot      → expr_prim

74)  expr_prim     → ( expressao )
75)  expr_prim     → chamada
76)  expr_prim     → id
77)  expr_prim     → num_int
78)  expr_prim     → num_dec
79)  expr_prim     → cadeia
80)  expr_prim     → true
81)  expr_prim     → false

```

Terminais e mapeamento com o léxico

Os terminais da gramática são os tipos de token produzidos pelo analisador léxico da Parte 1. Os quatro terminais genéricos abaixo carregam o lexema; os demais são as próprias palavras reservadas e símbolos, que casam diretamente com o `T o k e n T y p e` correspondente.

Terminal na GLC	Token (léxico)	Descrição
<code>id</code>	<code>IDENTIFICADOR</code>	Nome de variável ou função
<code>num_int</code>	<code>INTEIRO</code>	Literal inteiro (ex.: 42)
<code>num_dec</code>	<code>DECIMAL</code>	Literal decimal (ex.: 3.14)
<code>cadeia</code>	<code>STRING</code>	Literal entre aspas simples
<code>func main if else</code>	<code>(keywords)</code>	<code>for while switch case return int</code>
<code>float str bool true</code>	<code>(keywords)</code>	<code>false AND OR XOR XNOR puts input</code>
<code>() { } ; , : =</code>	<code>(pontuação)</code>	Delimitadores e atribuição
<code>+ - * / % ^</code>	<code>(operadores)</code>	Aritméticos
<code>= ! < > <= >=</code>	<code>(operadores)</code>	Relacionais
<code>\$</code>	<code>EOF</code>	Marca o fim da entrada

O token `ERRO`, emitido pelo léxico para caracteres inválidos, não é símbolo da gramática: ele dispara diretamente a rotina de tratamento de erros do parser.